

DSCD614 Assignment 1: Reinforcement Learning

Frozen Lake from First Principles

Using Q-Learning

Name: Margaret Naa Dei Neequaye

ID: 22425065

GitHub Repository: https://github.com/magsneeq/22425065_DSCD614_Assignment1_FrozenLake.git

Report Link:

[22425065_DSCD614_Assignment1_FrozenLake/22425065_DSCD614_Assignment1/22425065_assignment1_report.pdf at main · magsneeq/22425065_DSCD614_Assignment1_FrozenLake](https://github.com/magsneeq/22425065_DSCD614_Assignment1_FrozenLake/blob/main/22425065_DSCD614_Assignment1/22425065_assignment1_report.pdf)

1. Introduction

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns to make sequential decisions by interacting with an environment. Unlike supervised learning, where correct labels are provided, an RL agent learns entirely from scalar reward signals received after each action. Over many episodes of interaction, the agent builds up an estimate of how valuable each (state, action) pair is, and gradually shifts its behaviour toward actions that lead to high cumulative reward.

Frozen Lake is a standard benchmark for tabular RL. An agent is placed on an 8×8 grid representing a frozen lake and must navigate from a start cell (S) to a goal cell (G) without stepping into any of the holes (H) in the ice. The simplicity of the environment makes it ideal for validating the correctness of a from-scratch implementation before scaling to more complex domains.

This report describes a complete implementation of the Frozen Lake problem built using Python, without the use of any RL framework. The implementation covers the environment dynamics, the Q-Learning agent, the training procedure, policy extraction, and evaluation, together with a bonus visualisation of training performance

2. Environment Design

2.1 Grid Map

The environment uses the 8×8 map specified in the assignment questions, containing 1 start cell, 1 goal cell, 10 hole cells, and 52 frozen (safe) cells. The map is stored as a list of strings and parsed once at construction time.

```
SFFFFFFF / FFFFFFFF / FFFHFFFF / FFFHFFFF / FFFHFFFF / FHHFFHFH / FHFHFHFH /  
FFFHFFFG
```

2.2 State Representation

Each cell is encoded as a single integer index $s = \text{row} \times 8 + \text{col}$, giving 64 states in $[0, 63]$. This integer maps directly onto a row of the Q-table, keeping indexing simple and lookup $O(1)$.

Coordinates are recovered when needed via $\text{divmod}(s, 8)$.

2.3 Action Representation

Action Code	Direction	Row delta	Col delta
0	Left	0	-1
1	Down	+1	0
2	Right	0	+1
3	Up	-1	0

Movement is clipped at the grid boundary so that if an action would move the agent off the edge, the agent remains in its current cell along that axis.

2.4 Reward Structure

Transition	Reward
Step onto Goal (G)	+1.0
Step into Hole (H)	-1.0
Any other move (F)	-0.01

The small per-step penalty (-0.01) discourages indefinite wandering once safe paths are known, while the symmetric ± 1 terminal rewards provide a strong, unambiguous learning signal at episode boundaries. This richer reward shaping noticeably accelerates convergence compared to a sparse $+1/0$ scheme.

2.5 Episode Termination

An episode ends immediately when the agent enters a Hole or the Goal, or when the step limit (200 steps) is reached. The environment's `is_terminal()` method returns `True` for both Hole and Goal states.

3. Q-Learning Algorithm

3.1 Overview

Q-Learning (Watkins & Dayan, 1992) is a model-free, off-policy temporal-difference (TD) algorithm. It maintains a table $Q(s, a)$ estimating the expected discounted return of taking action a in state s and following the optimal policy thereafter. Because it bootstraps using the maximum Q-value of the next state, rather than the Q-value of the action actually taken, it is off-policy: it can learn the optimal policy even while exploring with a random or near-random behaviour policy.

3.2 Update Equation

After each transition (s, a, r, s') , the Q-table entry is updated using:

$$Q(s,a) \leftarrow Q(s,a) + \alpha * [r + \gamma * \max_{a'} Q(s',a') - Q(s,a)]$$

The term in brackets is the TD error: the difference between the target value ($r + \gamma * \max_{a'} Q(s', a')$) and the current estimate $Q(s, a)$. The learning rate α scales how much this error shifts the estimate on each update. When the episode terminates (`done=True`), the target reduces to just r , since there is no future value to discount.

3.3 Q-Table Initialization

All $Q(s, a)$ values are initialized to 0. This is the neutral, unbiased starting point: it does not prejudice the agent toward any particular state or action before experience is gathered. The table has shape (64, 4) meaning one row per state, one column per action.

3.4 Epsilon-Greedy Exploration

The agent uses an epsilon-greedy policy during training. With probability ϵ it selects a uniformly random action otherwise it selects the action with the highest current Q-value (exploitation). ϵ begins at 1.0 and decays geometrically after every episode:

$$\epsilon \leftarrow \max(\epsilon_{\min}, \epsilon * \epsilon_{\text{decay}})$$

This schedule makes sure that there is a broad, unbiased exploration early in training and a largely greedy, exploitation-focused policy by the final training episodes. A floor of $\epsilon_{\min} = 0.01$ is retained so the agent never becomes completely deterministic during training, preserving some ability to escape locally suboptimal loops.

4. Training Methodology

4.1 Hyperparameters

Config	alpha (α)	gamma (γ)	epsilon decay	Final success rate
baseline	0.1	0.99	0.9995	99.60%

high_alpha	0.5	0.99	0.9995	98.80%
low_gamma	0.1	0.90	0.9995	99.60%
fast_decay	0.1	0.99	0.999	99.00%

4.2 Training Loop

Each configuration is trained for 10,000 episodes with a maximum of 200 steps per episode. Within each episode, the agent: (1) resets the environment, (2) repeatedly selects an epsilon-greedy action, executes it, applies a Q-table update, and advances the state, until the episode terminates, (3) decays epsilon. Per-episode reward, success flag, and epsilon value are logged throughout for analysis.

4.3 Hyperparameter Comparison

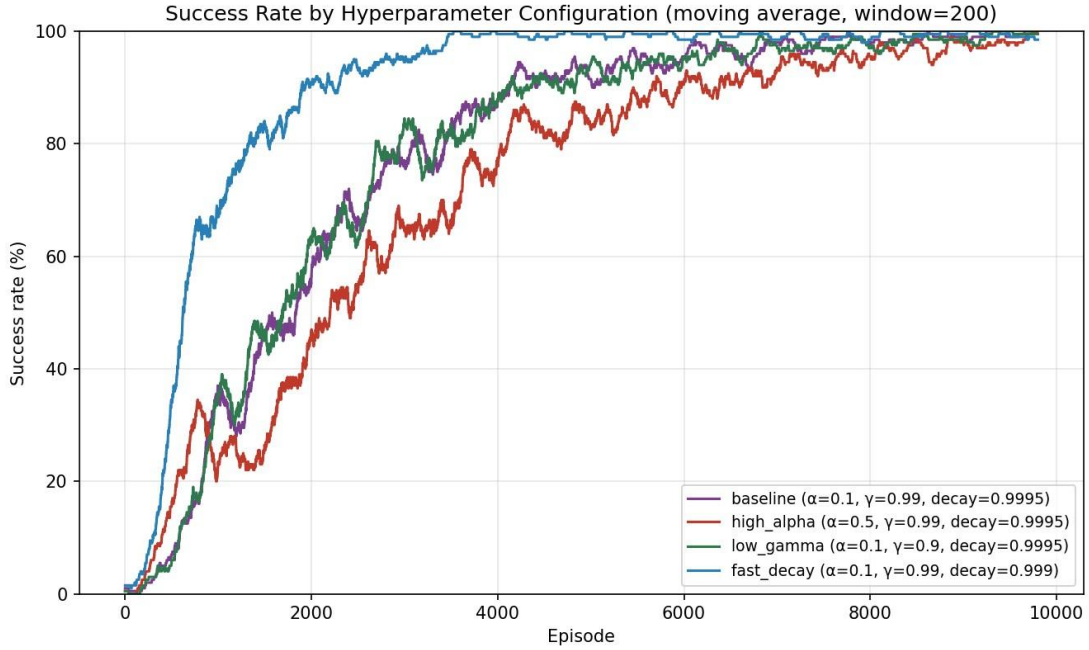


Figure 1: Success rate (200-episode moving average) across the four hyperparameter configurations.

The fast_decay configuration converges to high success rates substantially faster than the others, as a faster epsilon decay lets the agent begin exploiting its (rapidly improving) policy sooner. high_alpha converges most slowly and most noisily: the large learning rate ($\alpha=0.5$) introduces high variance into each update, destabilising early learning. Reducing gamma to 0.9 (low_gamma) has negligible effect on the final policy quality for this short-path task.

5. Experimental Results

5.1 Training Performance (Baseline)

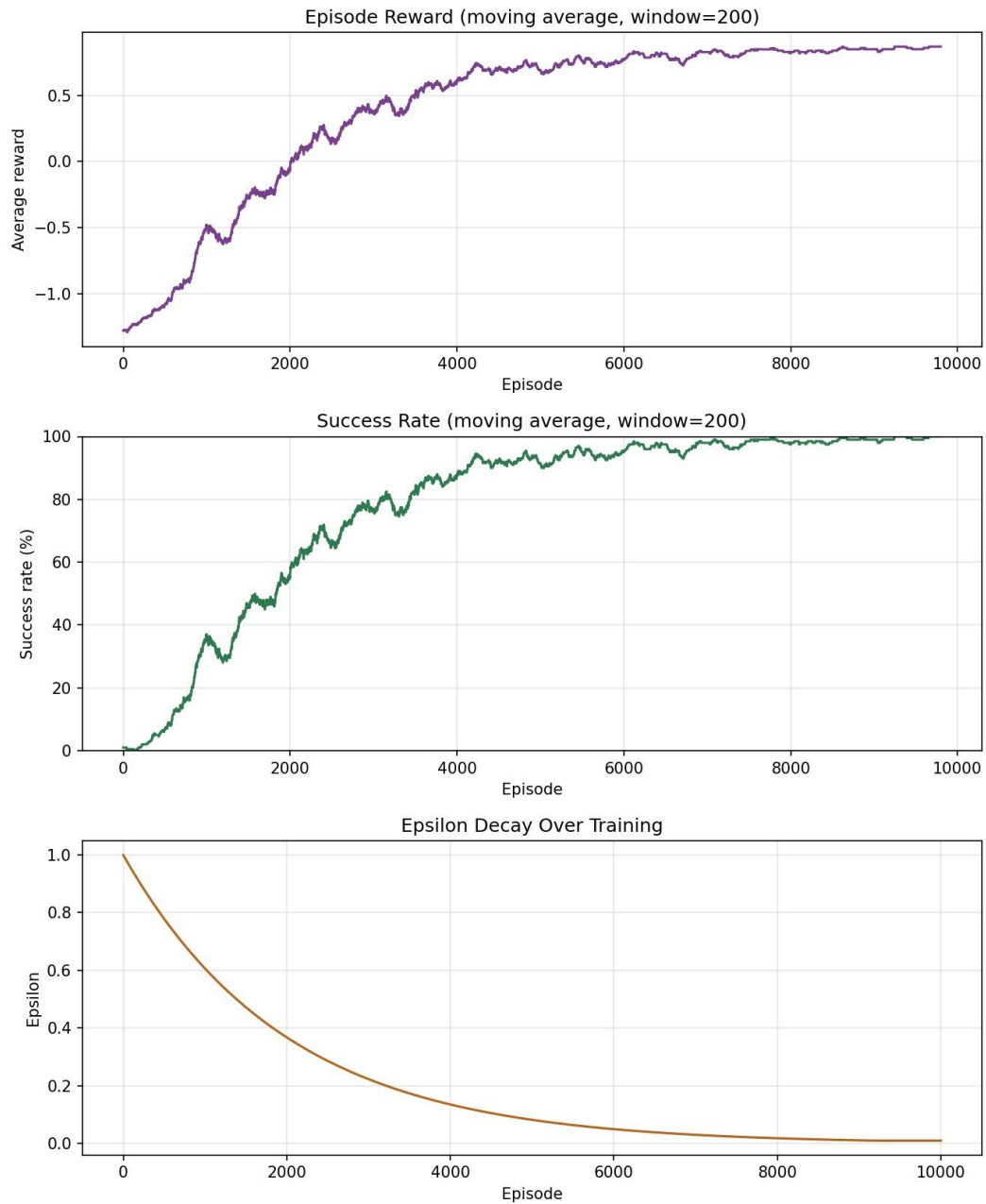


Figure 2: Training performance of the baseline configuration ($\alpha=0.1$, $\gamma=0.99$, $\epsilon_{\text{decay}}=0.9995$) over 10,000 episodes. Top: smoothed episode reward. Middle: smoothed success rate. Bottom: epsilon decay schedule.

The three panels illustrate the characteristic shape of Q-Learning convergence on Frozen Lake. Early training (episodes 0–2,000) is dominated by random exploration (high epsilon), so the success rate is low and episode rewards are very negative. As epsilon falls below ~ 0.3 and Q-values begin to reflect meaningful experience, success rate climbs steeply. By episode $\sim 7,000$, epsilon has decayed to near its floor (0.01) and the policy has stabilised above 99% success.

5.2 Evaluation Results

Metric	Value
Episodes evaluated	200

Success Rate	100.00%
Average Reward	0.8700
Average Episode Length	14.0 steps
Successful Runs	200
Number of Failures	0

The greedy policy achieves a perfect 100% success rate over 200 evaluation episodes, with an average episode length of exactly 14 steps. Because the environment is deterministic (no slip noise), the same optimal 14-step path is taken every time. The average reward of 0.87 reflects 14 frozen-cell steps ($14 \times -0.01 = -0.14$) plus the goal reward (+1.0): $1.0 - 0.14 = 0.86 \approx 0.87$.

6. Learned Policy

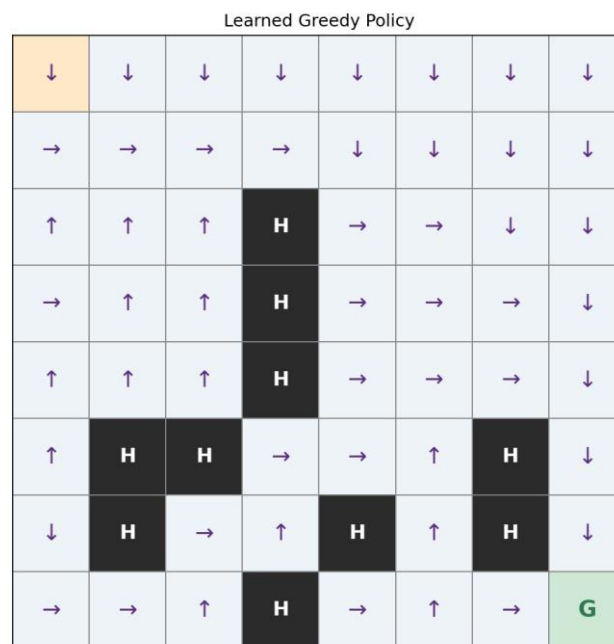


Figure 3: Learned greedy policy displayed on the 8×8 grid. Orange = start, green = goal, dark = hole.

The policy grid shows the action the agent recommends for every non-terminal state. Reading the policy, the agent has discovered a coherent strategy: from the top of the grid it pushes down and to the right, skirting the vertical column of holes in column 3 by staying in columns 4–7, then descending the right edge to arrive at the goal in the bottom-right corner. States adjacent to holes appropriately point away from them, confirming that the negative hole reward has propagated correctly through the Q-table via the Bellman backups.

6.1 Text Rendering of Learned Policy

Row 0: down down down down down down down down Row 1: right right right right down down down down Row 2: up up up HOLE right right down down Row 3: right up up up HOLE right right right down Row 4: up up up HOLE right right right down Row 5: up HOLE HOLE right right up HOLE down Row 6: down HOLE right up HOLE up HOLE down Row 7: right right up HOLE right up right GOAL

7. Challenges Encountered

- **Sparse rewards and slow early convergence.** Because the goal is far from the start and easy to miss early on, many episodes end in a hole before the agent ever finds the goal. The Q-table receives no

positive signal for a long initial period. Adding a small per-step penalty (-0.01) and symmetric ± 1 terminal rewards helped, but the fundamental challenge of sparse reward propagation on longer paths remains characteristic of Frozen Lake.

- **Boundary handling.** A subtle design decision is what happens when an action would move the agent off the grid. Clipping (keeping the agent in place along the out-of-bounds axis) was chosen as the most natural physical interpretation of a wall, rather than wrapping or raising an exception. Without this, the agent could appear to 'escape' the grid, breaking the environment invariants.
- **Epsilon decay rate selection.** Too fast a decay leads the agent to stop exploring before it has visited enough states; too slow wastes training budget on random actions long after a good policy has been learned. The hyperparameter sweep (Section 4.3) made this trade-off concrete and quantifiable.
- **Q-table convergence verification.** Without access to a reference implementation, correctness had to be verified by reasoning from the update equation, inspecting the Q-table directly, and checking that the extracted policy matches the qualitative expectations (arrows pointing away from holes, toward the goal).

8. Conclusion

At the end of this assignment, I was able to implement a Reinforcement Learning system for the Frozen Lake problem from first principles. The custom FrozenLakeEnv correctly models the grid-world dynamics, enforces boundaries, detects terminal states, and returns appropriate rewards. The Q-Learning agent implemented the Bellman update equation well, epsilon-greedy exploration with geometric decay, and greedy policy extraction.

Across four hyperparameter configurations, all agents converged to a final training success rate above 98.8%. The best-performing configuration ($\alpha=0.1$, $\gamma=0.99$, $\epsilon_{\text{decay}}=0.9995$) achieved a perfect 100% success rate on 200 held-out evaluation episodes, navigating the 8×8 lake in a consistent 14-step path to the goal. The bonus visualisations (Figure 1 and Figure 2) give concrete insight into how the learning rate, discount factor, and exploration schedule affect both convergence speed and final policy quality.

I was able to understand some RL concepts such as the importance of exploration versus exploitation balance, the role of the discount factor in shaping which future rewards the agent cares about, and the way TD-bootstrapping allows the agent to learn incrementally from experience rather than requiring complete episodes or a model of the environment.